Smart LLM Router: A Free-First, Cost-Aware Gateway for Routing Coding Assistance Between Free and Paid Language Models

Abhilash Shishodia

Independent Researcher

July 14, 2026

Abstract

Premium AI coding assistants built on frontier large language models (LLMs) deliver excellent quality but charge per token, so cost scales with usage even though a large fraction of everyday developer requests—autocompletion, boilerplate generation, and short explanations—do not require a frontier model. We present the *Smart LLM Router*, a free-first, cost-aware gateway that classifies each incoming request, serves it with a free or local model by default, and escalates to a paid model only when the task is predicted to be hard or when a lightweight verifier judges the free response to be inadequate. The router exposes a drop-in, OpenAI-compatible interface, so existing IDE and CLI clients need no changes. We formalize the routing problem as constrained cost minimization subject to a quality floor, describe a *classify–route –verify* pipeline with a confidence-based escalation loop, and present an analytic cost model showing that expected spend is proportional to the fraction of requests that ultimately reach a paid tier. We report a preliminary empirical study on a 44-problem coding suite with executable unit tests, using *small locally-running (free) models* and *large cloud-based models* that are comparable in capability to paid-tier hosted models; evaluating true metered paid APIs is left as future work. On the hardest problems small local models solve far fewer tasks than large models, yet the router—by keeping easy requests free and escalating only when a verifier flags a failure—matches the strong model's task-success rate while sending only a fraction of requests to the large tier. We also provide a reproducible evaluation protocol (datasets, metrics, baselines, and ablations) for larger-scale validation. We situate the design against existing routers and gateways (e.g., RouteLLM, LiteLLM, OpenRouter, Not Diamond) and highlight its distinguishing combination of free-first defaults, automatic escalation, local-privacy routing, and per-request budget guardrails.

Keywords: LLM routing, model cascades, cost-aware inference, AI gateway, code assistants, model selection.

1 Introduction

Large language models (LLMs), built on the transformer architecture [20], have become a standard part of the software-development workflow, powering code completion, refactoring, test generation, debugging, and natural-language explanation of code. The strongest of these assistants are backed

by frontier models offered under metered, per-token pricing. While the quality is high, the cost structure is unfavorable for teams: spend grows linearly with usage, bills are difficult to predict, and—crucially—the *same* premium price is paid for trivial and hard requests alike.

Two empirical observations motivate this work. First, the difficulty distribution of real developer requests is highly skewed: many requests are easy (rename a symbol, explain a function, generate a small stub) and a minority are genuinely hard (design a scalable architecture, reason about a subtle concurrency bug, perform a security-sensitive refactor). Second, the capability gap between free/open and premium models has narrowed substantially for easy tasks, so a cheaper model often produces an answer indistinguishable from a frontier model on such requests. Together these imply that routing every request to a frontier model overspends on the easy majority.

We propose the **Smart LLM Router**: a gateway that sits between the client (IDE or CLI) and a pool of model providers and *automatically decides, per request, whether a free/local model suffices or a paid model is warranted*. The router is *free-first*: it defaults to zero-marginal-cost models and escalates to paid tiers only on demand. Escalation is driven both by a *predictive* signal (a complexity classifier applied before generation) and a *reactive* signal (a verifier that grades the free answer and triggers re-routing when confidence is low). The interface is OpenAI-compatible, so adoption requires no client changes.

Contributions.

- We formalize free-first coding-assistant routing as constrained cost minimization subject to a quality floor (Section 4).
- We describe a *classify–route–verify* architecture with an invisible escalation loop and per-request budget guardrails (Sections 3–4).
- We derive an analytic cost model that expresses expected spend and relative savings in terms of the effective paid-routing fraction (Section 5).
- We contribute a concrete, reproducible evaluation protocol for software-engineering workloads (Section 7).
- We position the design against existing routers and gateways and articulate what it uniquely combines (Section 2).

2 Background and Related Work

Model cascades and cost-aware inference. Routing simpler queries to cheaper models is an established idea. FrugalGPT [2] introduced LLM cascades that call models in increasing order of cost and stop early when a scorer deems an answer adequate. RouteLLM [1] learns a binary router between a strong and a weak model from human-preference data, reporting up to 85% cost reduction while retaining roughly 95% of GPT-4 quality on common benchmarks, and provides a drop-in OpenAI-compatible server with threshold calibration. Hybrid LLM [3] similarly learns quality-aware query routing between a small and a large model. AutoMix [4] mixes models using self-verification to decide when to escalate. Our design draws on both the predictive routing of RouteLLM/Hybrid LLM and the reactive, verify-then-escalate cascade of FrugalGPT/AutoMix, but frames them within a *free-first*, multi-tier, privacy- and budget-aware gateway targeted at coding workloads.

Table 1: Representative routers and gateways. "Quality-aware" means routing uses a learned or verified estimate of answer quality rather than only static rules.

System	Type	Hosting	Quality-aware	Open source
RouteLLM [1]	Router framework	Self-host	Yes	Yes
FrugalGPT [2]	Cascade	Self-host	Yes	Partial
Hybrid LLM [3]	Router	Self-host	Yes	Partial
LiteLLM [12]	Gateway	Self-host	No	Yes
OpenRouter [13]	Unified API/router	Hosted	Partial	No
Portkey [14]	Gateway	Both	Partial	Partial
Not Diamond [15]	Router	Hosted	Yes	No
Semantic Router [18]	Decision layer	Self-host	No	Yes
Smart LLM Router (ours)	Free-first gateway	Self-host	Yes	—

Gateways and hosted routers. LiteLLM [12] is an open-source AI gateway exposing an OpenAI-compatible API to 100+ providers with fallbacks, load-balancing, and spend tracking, but its routing is primarily rule- and fallback-based rather than quality-aware. OpenRouter [13] is a hosted unified API to many models (including free ones) with automatic and price-based routing. Portkey [14] is an AI gateway with routing, caching, guardrails, and observability. Not Diamond [15], Martian [16], and Unify [17] are hosted services that select a model per query to optimize cost, quality, and latency. Semantic Router [18] is an open-source decision layer that routes by embedding similarity (intent), not by predicted difficulty or cost.

Local serving. Free-first routing is enabled by efficient local inference. Systems such as vLLM [5] and packagers such as Ollama [19] make it practical to serve capable open models (e.g., code-specialized models) on local hardware at zero marginal cost and with data locality.

Code benchmarks. Our evaluation follows the lineage of standard code-generation benchmarks that grade models by executing generated programs against unit tests: HumanEval [6] and MBPP [7] for function-level synthesis, BigCodeBench [10] for more realistic library usage, LiveCodeBench [9] for contamination-resistant, continuously updated problems, and SWE-bench [8] for repository-level issue resolution. We adopt the same execution-based, *pass@1* methodology on a compact suite spanning five difficulty tiers (Section 7).

Positioning. Table 1 summarizes representative systems. Relative to prior work, the Smart LLM Router's distinguishing feature is not a single new mechanism but a *combination*: free/local defaults, *both* predictive and reactive escalation to paid tiers, explicit privacy-based routing of sensitive code to local-only models, and hard per-request/per-period budget guardrails—delivered behind one drop-in gateway.

3 System Design

The router is a stateless service exposing an OpenAI-compatible `/chat/completions` endpoint. Each request passes through a short *classify–route–verify* pipeline (Figure 1).

1. **Classify.** A fast complexity estimator maps the request (prompt, task type, context size, attached files) to a scalar difficulty score $x(q) \in [0, 1]$. The estimator can be rule-based, embedding-based, or a small fine-tuned classifier; these can be combined.

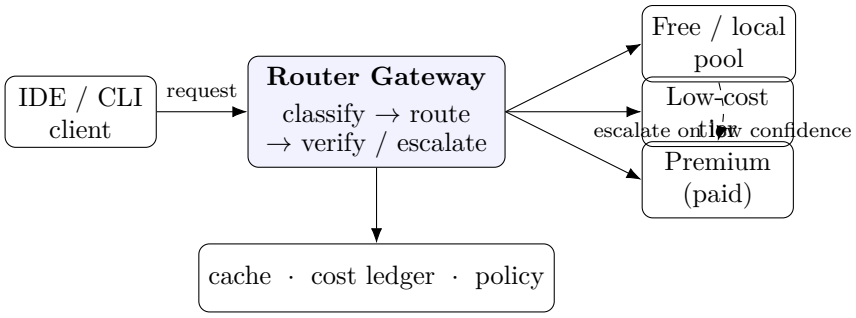

Figure 1: The Smart LLM Router. Requests are classified, routed to the cheapest adequate tier, and—if the free/low-cost answer fails verification— transparently escalated to a paid tier. Auxiliary services provide caching, cost accounting, and policy.

2. **Route.** A policy π selects the cheapest model tier whose predicted quality satisfies the caller's quality floor, subject to privacy and budget constraints.
3. **Verify.** For a request served by a lower tier, a lightweight verifier $v(\cdot)$ grades the response. If confidence is below a threshold δ, the request is transparently re-routed (escalated) to the next tier. The client observes only the final answer.

Auxiliary services support the pipeline: a response *cache* avoids paying twice for identical requests; a *cost/usage ledger* records per-request spend for budgeting and analytics; and a *policy configuration* declares thresholds, budgets, and privacy rules.

4 The Decision Engine

4.1 Problem formulation

Let a query be q drawn from workload distribution $\mathcal{D}$. Let the model pool be organized into K tiers $T_1, \ldots, T_K$ ordered by increasing cost, with per-request cost c_k (with $c_1 = c_{\text{free}} \approx 0$ for the free/local tier) and a quality function $Q_k(q) \in [0, 1]$ giving the (expected) quality of tier k on query q. A router is a policy π that maps observable request features to a tier (and possibly a sequence of tiers via escalation). We seek to minimize expected cost subject to a quality floor Q_{target}:

$$\min_{\pi} \ \mathbb{E}_{q\sim\mathcal{D}}\big[\, C_\pi(q) \,\big] \quad \text{s.t.} \quad \mathbb{E}_{q\sim\mathcal{D}}\big[\, Q_\pi(q) \,\big] \ \geq \ Q_{\text{target}}, \tag{1}$$

where $C_\pi(q)$ and $Q_\pi(q)$ are the realized cost and quality of the policy on q. Introducing a multiplier $\lambda \geq 0$ yields the unconstrained surrogate

$$\min_{\pi} \ \mathbb{E}_{q}\big[\, C_\pi(q) \ - \ \lambda\, Q_\pi(q) \,\big], \tag{2}$$

so that λ trades cost against quality; it is the analogue of RouteLLM's calibrated cost threshold [1].

4.2 Predictive routing signals

Before generation, the router combines several signals into the difficulty score $x(q)$ and side constraints:

- **Task type.** Autocompletion, boilerplate, and short explanations bias toward free tiers; architecture, security, and multi-file debugging bias toward paid tiers.

- **Predicted complexity.** A classifier or heuristics estimate difficulty from the prompt and repository context.
- **Context size.** Very long contexts favor a capable tier able to use them effectively.
- **Privacy flag.** Requests touching sensitive code set a hard constraint $\pi(q) \in \{\text{local tiers}\}$.
- **Budget guardrail.** A remaining-budget state b caps paid usage over a period; when b is exhausted, paid tiers are disabled.

For a two-tier (free/paid) instantiation with threshold τ, the predictive rule is

$$r_{\text{pred}}(q) = \begin{cases} \text{paid}, & x(q) \geq \tau \ \text{ and privacy/budget allow}, \\ \text{free}, & \text{otherwise}. \end{cases} \tag{3}$$

4.3 Reactive escalation

When a request is served by the free tier, the verifier produces a confidence score $v(a, q) \in [0, 1]$ for answer a (e.g., self-consistency, unit-test execution for code, a small critic model, or format/lint checks). If $v(a, q) < \delta$, the request escalates to the paid tier. Escalation is invisible to the client. The verifier is the crux of the system: an over-eager verifier wastes escalations, while a lax one degrades quality.

5 Cost Model

Consider the two-tier instantiation with $c_{\text{free}} \approx 0$. Let

$$p \;=\; \underbrace{\Pr\big[x(q) \geq \tau\big]}_{\text{routed paid directly}} \;+\; \underbrace{\Pr\big[x(q) < \tau\big]\ \Pr\big[v(a, q) < \delta \mid \text{free}\big]}_{\text{escalated after verify}} \tag{4}$$

be the *effective paid fraction*: the probability that a request ultimately consumes the paid tier. Ignoring the small cost of local generation and verification, the expected cost per request is

$$\mathbb{E}[C] \;=\; p\, c_{\text{paid}}, \tag{5}$$

whereas the all-paid baseline costs c_{paid} per request. The relative saving is therefore

$$\text{Savings} \;=\; 1 - p. \tag{6}$$

Equation (6) makes the design goal explicit: *minimize the effective paid fraction p without violating the quality floor*. Lowering τ or δ reduces p (more savings) but risks quality; the multiplier λ in Eq. (2) sets the operating point.

For the general K-tier cascade with escalation probabilities, the expected cost telescopes as

$$\mathbb{E}[C] \;=\; \sum_{k=1}^{K} c_k\, \rho_k, \qquad \rho_k \;=\; \Pr[\text{tier } k \text{ is invoked}], \tag{7}$$

where ρ_k is determined by the routing thresholds and the per-tier escalation rates. A worked illustration: if 75% of requests are served for free and never escalate, and the remaining 25% reach the paid tier at $c_{\text{paid}} \approx \$0.03$ per request, then $p = 0.25$ and expected spend is \$0.0075 per request—about a 75% reduction versus all-paid. These figures are illustrative; realized savings depend on the workload's difficulty distribution and verifier accuracy, which motivates the evaluation below.

Table 2: Execution-based pass@1 on the 44-problem suite (higher is better). Small local free models match large models on easy work but lag sharply on the hardest tier, which is exactly what a difficulty-aware router can exploit.

Model	Class	Overall	Easy	Med.	Hard	Exp.	Brutal
`llama3.2` (3B)	small local free	73%	100%	100%	100%	58%	42%
`gemma3`	small local free	93%	100%	100%	88%	100%	83%
`phi4-mini`	small local free	52%	100%	88%	75%	33%	17%
`gpt-oss:120b`	large cloud	100%	100%	100%	100%	100%	100%
`qwen3-coder:480b`	large cloud	98%	100%	100%	100%	100%	92%
avg small local free		73%					47%
avg large cloud		99%					96%

6 Empirical Evaluation

We report a preliminary study that instantiates the router and measures both the capability gap it exploits and its end-to-end behavior.

6.1 Setup

We implement the gateway as a stateless service backed by Ollama [19]. We evaluate two model classes, using deliberately honest labels:

- **Small local free models** run locally at zero marginal cost and are always available offline: `llama3.2` (3B), `gemma3`, and `phi4-mini`.
- **Large cloud-based models** are large models served from Ollama's cloud that are comparable in capability to paid-tier hosted models: `gpt-oss:120b` and `qwen3-coder:480b`. We do *not* evaluate metered commercial paid APIs; extending the study to true paid tiers is future work.

Our workload is a suite of 44 Python programming problems spanning five difficulty tiers (easy, medium, hard, expert, and "brutal"), each with hidden unit tests. We grade with execution-based *pass@1*: a task counts as solved if the model's generated function passes all its tests, run in an isolated subprocess with a timeout. In these experiments the reactive verifier executes the task's unit tests, i.e., it is an *oracle* verifier; realistic deployment verifiers (generated tests, self-consistency, critic models) are weaker, so our escalation results are an *upper bound* on what reactive verification recovers. Reported dollar figures use the configurable, illustrative pricing of Section 5, not vendor prices.

6.2 Capability gap between model classes

Table 2 reports per-tier pass@1 for each model on the full suite. The pattern that motivates free-first routing is clear: on easy and medium problems the small local models are competitive with the large models, but on the hardest ("brutal") tier they fall well behind—averaging 47% versus 96% for the large cloud models—so difficulty-aware routing has real headroom.

Table 3: Router versus baselines on the 44-problem suite (small local tier `llama3.2`; large cloud tier `gpt-oss:120b`; τ=0.45). The router matches the large model's pass@1 while sending only 52% of requests to the large cloud tier. Cost uses the illustrative pricing of Section 5, not vendor prices.

Strategy	pass@1	Cloud frac. p	Cost	Avg. latency	Savings
Always free (local)	80%	0%	\$0.0000	2.41 s	100%
Always cloud (baseline)	100%	100%	\$0.0491	4.06 s	0%
Random	84%	41%	\$0.0194	3.04 s	61%
Router (ours)	**100%**	52%	\$0.0287	3.80 s	**41%**

6.3 Router effectiveness

We run the four strategies of Section 4 on the same suite with `llama3.2` as the small local (free) tier and `gpt-oss:120b` as the large cloud tier, threshold τ=0.45. Table 3 summarizes the outcome. Always-free solves only the easy majority; always-cloud solves everything at full cost. The router *matches the large model's task-success rate* while routing a substantial share of requests to the free tier, because the reactive (oracle) verifier catches cases the predictive classifier under-rated and escalates only those. Consistent with Eq. (6), realized cost tracks the effective paid fraction p. Six requests were escalated after a silent local failure (e.g., regular-expression matching, the parenthetical calculator, and decode-string), and all six were recovered by the large model—directly illustrating the value of reactive verification.

7 Proposed Larger-Scale Evaluation

Our study above is small and uses an oracle verifier; we outline a fuller protocol for community-scale validation.

Datasets and workloads. (i) Code generation: HumanEval [6] and MBPP [7]; (ii) repository-level tasks: SWE-bench [8]; (iii) general instruction quality: MT-Bench [11]; (iv) a de-identified trace of real developer requests to reflect the true difficulty distribution and task mix.

Metrics. For each system we report: expected cost per request and total spend; the effective paid fraction p; task quality (*pass@k* for code; LLM-judge win-rate versus a frontier reference for open-ended tasks); escalation rate and escalation precision/recall; added end-to-end latency (routing plus verification overhead); and cache hit rate.

Baselines. Always-paid (upper cost bound), always-free (lower cost bound), random routing, RouteLLM [1] (mf router), and a FrugalGPT-style cascade [2]. Reporting cost–quality curves (quality versus spend as τ, δ vary) enables Pareto comparison rather than single-point claims.

Ablations. (i) classifier versus pure heuristics for $x(q)$; (ii) with and without the reactive verifier; (iii) verifier variants (unit-test execution, self-consistency, critic model); (iv) effect of budget guardrails; and (v) sensitivity to the free-tier model choice.

Hypotheses. H1: free-first routing attains a strictly better cost–quality Pareto frontier than always-paid on realistic coding traces. **H2:** reactive verification recovers most of the quality lost

by aggressive predictive routing at modest additional cost. **H3:** the dominant cost driver is the effective paid fraction p, as predicted by Eq. (6).

8 Discussion, Limitations, and Ethics

Limitations. The system's savings and quality hinge on the verifier and classifier; both can be miscalibrated or drift as workloads change, requiring periodic recalibration. Routing and verification add latency, which must stay small relative to generation time. Free API tiers impose rate limits and may change terms; local models require adequate hardware. Benchmark contamination and the non-stationarity of hosted models complicate reproducibility; we therefore emphasize cost–quality *curves* and pinned model versions.

Ethical and practical considerations. Free-first routing can lower the cost and energy footprint of AI assistance and, via local-only routing of sensitive code, improve data locality and privacy. Conversely, mis-routing risks silently degrading answer quality; transparent logging of which tier served each request, along with user-visible controls, mitigates this. Budget guardrails prevent runaway spend but must not be used to degrade service without disclosure.

9 Conclusion

We presented the Smart LLM Router, a free-first, cost-aware gateway that routes coding-assistance requests to free or local models by default and escalates to paid models only when predicted difficulty or verified low confidence warrants it. We formalized routing as constrained cost minimization, described a classify–route–verify architecture with transparent escalation, derived a cost model in which expected spend is proportional to the effective paid fraction, and proposed a reproducible evaluation protocol. We hope this design and protocol help teams obtain most of the benefit of frontier coding assistants at a fraction of the cost, while retaining privacy and predictable budgets.

References

[1] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv:2406.18665*, 2024. `https://arxiv.org/abs/2406.18665`

[2] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv:2305.05176*, 2023. `https://arxiv.org/abs/2305.05176`

[3] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, and A. H. Awadallah. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. *International Conference on Learning Representations (ICLR)*, 2024. `https://arxiv.org/abs/2404.14618`

[4] A. Madaan, P. Aggarwal, A. Anand, et al. AutoMix: Automatically Mixing Language Models. *arXiv:2310.12963*, 2023. `https://arxiv.org/abs/2310.12963`

[5] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *ACM Symposium on Operating Systems Principles (SOSP)*, 2023. `https://arxiv.org/abs/2309.06180`

[6] M. Chen, J. Tworek, H. Jun, et al. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021. `https://arxiv.org/abs/2107.03374`

[7] J. Austin, A. Odena, M. Nye, et al. Program Synthesis with Large Language Models. *arXiv:2108.07732*, 2021. https://arxiv.org/abs/2108.07732

[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024. https://arxiv.org/abs/2310.06770

[9] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code. *arXiv:2403.07974*, 2024. https://arxiv.org/abs/2403.07974

[10] T. Y. Zhuo, M. C. Vu, J. Chim, et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv:2406.15877*, 2024. https://arxiv.org/abs/2406.15877

[11] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023. https://arxiv.org/abs/2306.05685

[12] BerriAI. LiteLLM: Python SDK and Proxy Server (AI Gateway) to call 100+ LLM APIs in OpenAI format. Software, accessed 2026. https://github.com/BerriAI/litellm

[13] OpenRouter. A unified API and marketplace for LLMs with model routing. Accessed 2026. https://openrouter.ai

[14] Portkey. AI Gateway: routing, caching, guardrails, and observability for LLMs. Accessed 2026. https://portkey.ai

[15] Not Diamond. An AI model router that selects the best model per query. Accessed 2026. https://www.notdiamond.ai

[16] Martian. Model router for cost- and quality-aware LLM selection. Accessed 2026. https://www.withmartian.com

[17] Unify. Routing and benchmarking across LLM providers via a single API. Accessed 2026. https://unify.ai

[18] Aurelio AI. Semantic Router: a fast decision layer for LLMs and agents. Software, accessed 2026. https://github.com/aurelio-labs/semantic-router

[19] Ollama. Run open large language models locally. Accessed 2026. https://ollama.com

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762